

소재정보학

이예리

한국화학연구원 디지털화학연구센터

yealee@kriict.re.kr

강의개요 – 2026 가을 UST 소재정보학

교과목	소재정보학
강사진	한국화학연구원 화학데이터기반연구센터 안광노, 우준희, 유수현, 이예리, 장승훈
학점	3학점
성적평가 방법	Grade (등급제)
성적평가 세부항목	출석 20%, 중간고사 시험 30%, 개인 프로젝트 50%
강의 요일 및 시간	매주 목요일 오후 2시 ~ 5시
수업 방식	유인물을 이용한 강의 및 컴퓨터 실습 진행

주별	내용	비고
1주차	과목 소개 및 소재정보학 개론	이예리
2주차	기계학습 기초 1 (기초 수학, 다양한 기계학습 방법, 지도학습, 비지도학습)	장승훈
3주차	기계학습 기초 2 (기초 수학, 다양한 기계학습 방법, 지도학습, 비지도학습)	장승훈
4주차	플랫폼 기반 소재 인공지능 소개 및 실습	장승훈
5주차	소재 탐색 사례 소개 및 실습	이예리
6주차	딥러닝 기초, 그래프 표현	이예리
7주차	소형 AI 에이전트 구축 실습을 통한 SISSO 기법 활용한 소재 물성 예측	유수현
8주차	중간고사	이예리
9주차	소재 탐색에서의 베이지안 최적화의 활용	안광노
10주차	소재 탐색을 위한 실험 자동화 인터페이싱 기초	안광노
11주차	실험 공정 설계를 위한 언어모델 및 RAG의 활용	안광노
12주차	LLM 기반 소재 데이터 수집	우준희
13주차	온톨로지 기반의 소재 데이터베이스 구축 기초	유수현
14주차	AI, 로봇을 활용한 자율 실험 기반의 신소재 탐색 연구	우준희
15주차	자율화 실험실용 온톨로지 기반의 플랫폼 구축 및 활용	유수현
16주차	기말 프로젝트 발표	이예리

강의 홈페이지

education > 2026-Materials-Informatics > Wiki > Home

Last edited by  Yea-Lee Lee right now

Page history

New page

Home



소재정보학 특론 (UST, 2026년 가을학기)

☐ 강사진: 한국화학연구원 디지털화학연구센터 (안광노, 우준희, 유수현, 이예리, 장승훈)

☐ 학점: 3학점

☐ 성적평가방법: Grade (등급제)

☐ 성적평가 세부항목: 출석 20%, 중간고사 시험 30%, 개인 프로젝트 50%

☐ 강의 요일 및 시간: 매주 목요일 오후 2시 ~ 5시

☐ 강의실: 한국표준과학연구원 301동(신소재동) 지하 1층 회의실 1

☐ 수업 방식 : 유인물을 이용한 강의 및 컴퓨터 실습 진행

☐ 기말고사 프로젝트 예시

1주차

과목 소개 및 소재정보학 개론

2025.09.03 이예리

표준연 301동 지하1층 회의실 1

2주차

기계학습 기초 1

2026.09.10 장승훈

화학연 디딤돌플라자 소회의실(304호-12석)

3주차

기계학습 기초 2

2025.09.17 장승훈

화학연 디딤돌플라자 소회의실(309호-12석)

4주차

플랫폼 기반 소재 인공지능 소개 및 실습

2025.09.24 (보강예정) 장승훈

화학연 디딤돌플라자 소회의실(304호-12석)

5주차

소재 탐색 사례 소개 및 실습

2025.10.01 이예리

화학연 디딤돌플라자 소회의실(304호-12석)

6주차

딥러닝 기초, 그래프 표현

2025.10.08 이예리

화학연 디딤돌플라자 소회의실(304호-12석)

7주차

소형 AI 에이전트 구축 실습을 통한 SISSO 기법 활용한 소재 물성 예측

2025.10.15 유수현

화학연 디딤돌플라자 소회의실(304호-12석)

- Gitlab 강의 자료실
 - <https://gitlab.chemdx.org/education/2026-materials-informatics/-/wikis/home>
- 강의자료 (유인물 및 실습자료) 매주 업로드 예정

강의 구성

머신러닝 알고리즘
(기초)

기초 수학, 다양한 기계학습 방법,
지도학습, 비지도학습
딥러닝 기초, neural network

소재 표현자

화학원소 특성값, 그래프, SMILES 등
SISSO 기법, 베이지안 최적화 활용

소재 데이터베이스

LLM 기반 소재 데이터 수집,
온톨로지 개념 및 활용

머신러닝 알고리즘
(고급)

자율실험실 기반 소재 탐색,
LLM 및 RAG 활용 실험 설계

평가 항목

- 출석 (20)
- 중간고사 (30) – 지필고사 (간단한 개념)
- 기말고사 (50) – 개인 프로젝트

순번	주제	설명
1	Au 클러스터 데이터베이스 구축	교육 시간에 배우는 PyXtal을 이용해서 원자 개수 N개의 Au 나노입자들을 제작합니다. (여기서 N은 4 이상의 자연수입니다.) 또한 수업 시간에 배우는 ASE의 EMT 계산 방법을 활용해서 각 구조들을 최적화합니다. 계산 결과 얻을 수 있는 총에너지, 원자 사이 결합 길이, 대칭 등의 정보를 수집하고, Pandas를 이용해서 정리합니다. 구조 최적화 이후에 대칭 정보가 바뀌었는지도 정리합니다. 또한 소재 정보학 융합 교육에서 배운 다양한 기법들을 이용하여 정보를 분석합니다.
2	공공데이터 포털 데이터 다운로드 및 분석	공공데이터에서 국민건강보험공단 건강검진정보를 다운받습니다. (예: https://www.data.go.kr/data/15007122/fileData.do) 건강검진정보 사이 상관관계를 분석하고 시각화합니다. 특히 흡연상태나 음주여부와 같은 생활 습관이 건강에 미치는 영향을 분석합니다. 과거 데이터와의 비교를 통해 건강 지표들이 악화 되었는지 아닌지를 분석합니다. 또한 재 정보학 융합 교육에서 배운 다양한 기법들을 이용하여 정보를 분석합니다.
3	AI를 이용한 분자의 HOMO-LUMO gap 예측	분자의 HOMO와 LUMO 사이의 gap은 분자가 얼마만큼의 에너지를 흡수하고 방출하는지를 결정하는 아주 중요한 물성이다. HOMO-LUMO gap 값은 새로운 형광 분자 설계 및 형광 소자 설계 등에 중요하게 사용되는 값이다. 본 프로젝트에서는 DFT를 이용해서 예측된 HOMO-LUMO gap 데이터에 기반하여, DFT 값을 근사하는 모델을 학습하는 것을 목표로 한다. (* 데이터 제공 예정)
4	AI를 이용한 분자의 약물 유사도 예측	인공지능을 이용한 신약 개발 분야에서 중요한 주제 중의 하나는 약물이 될 수 있는 분자를 구별할 수 있는 기준, 함수를 만드는 것이다. 본 프로젝트에서는 약물과 약물이 아닌 분자의 데이터에 기반을 두고 두 가지를 구별할 수 있는 classifier를 생성하는 것을 목표로 한다. 약물과 비약물을 구별하는 문제의 어려운 점은 두 가지 타입의 데이터의 개수가 매우 많이 차이가 난다는 점이다. 이러한 데이터 개수의 차이를 효과적으로 고려하여 정확하게 약물이 될 가능성이 높은 분자를 찾아낼 수 있는 모델 또는 약물 유사도를 점수로 표시할 수 있는 모델을 만들어 본다. (* 데이터 제공 예정)

평가 항목

- 출석 (20)
- 중간고사 (30) – 지필고사 (간단한 개념)
- 기말고사 (50) – 개인 프로젝트

5	원자특성 기반 금속/비금속 분류모델 생성	참고문헌(https://pubs.acs.org/doi/abs/10.1021/acs.jpcclett.8b00124)의 서포팅 자료에서 밴드갭 데이터셋(jz8b00124_si_002.xlsx)을 다운로드 받습니다. 교육시간에 배우는 SimPL-ML (www.simpl-ml.org) 플랫폼을 이용하여 원자특성 기반 feature-set을 생성합니다. sklearn에서 제공하는 SGD 분류기를 사용하여 금속/비금속 이진분류모델을 만들고 오차 행렬 (confusion matrix)을 조사합니다. 결정임계값에 따른 정밀도와 재현율 곡선을 그려보고, 추가적으로 ROC 곡선도 그려봅니다. RandomForest 분류기를 통해서도 분류모델을 만들어보고 SGD 분류기의 결과와 비교합니다.
6	제일원리 계산을 통한 분자 선별	G2 set에 대해서 제일원리 밀도범함수 이론 계산을 수행하고 결과 데이터를 분석한다. G2 set은 Atomic Simulation Environment (ASE)에 저장되어 있으며, GPAW 코드를 이용하여 제일원리 계산을 수행한다. Linear Combination of Atomic Orbital (LCAO) 계산을 수행하며, 구조 최적화를 하지 않는다면 일반적인 개인 랩탑에서도 충분히 짧은 시간 안에 계산 가능하다. HOMO-LUMO gap이나 magnetic moment, dipole moment와 같은 값을 얻는다. 교환-상관(exchange-correlation)을 LDA나 PBE, RPBE 등으로 설정하여 차이를 확인할 수 있다. 계산된 데이터를 Pandas를 이용하여 최대값, 최소값, 평균값, 상관관계 등을 시각화하여 분석하고, csv나 xlsx 파일 포맷으로 저장한다.
7	Chromophore 분자의 형광특성 예측	참고문헌(https://www.nature.com/articles/s41597-020-00634-8)에서 Chromophore와 Solvent에 대한 형광 특성DB를 다운로드받습니다. RDKit 파이썬 모듈을 활용하여 Chromophore와 Solvent 분자들에 대한 Fingerprint를 생성합니다. 교육시간에 배우게 될 다양한 머신러닝 예측 알고리즘을 활용하여 Chromophore 분자의 Absorption max, Emission max, Lifetime 예측을 수행합니다. Chromophore와 Solvent 분자들 사이의 상호작용이 고려된 모델이 만들어졌는지 테스트해보고, 예측모델에 따른 예측성능 비교도 진행해봅니다. (* 데이터 제공 예정)

평가 항목

- 출석 (20)
- 중간고사 (30) – 지필고사 (간단한 개념)
- 기말고사 (50) – 개인 프로젝트

12	무기 형광체 광학 특성 예측	https://figshare.com/articles/dataset/Optical_property_database_of_inorganic_phosphor_IPOP_dataset/21766916 에서 무기 형광체에 대한 광학특성 데이터셋(IPOP dataset)을 다운로드 받습니다. Pandas, Matplotlib python library를 활용하여 각 광학특성별 데이터 분포에 대한 가시화 및 분석을 수행해봅니다. 소재 정보학 초급 강의(지도학습)를 통해서 소개된 소재 인공지능 플랫폼, SimPL-ML 또는 ChemAI를 활용하여 무기 형광체의 여러 광학 특성들을 예측해봅니다. 물론, scikit-learn python library를 활용하여 직접 코딩을 통해 광학특성 예측을 수행할 수도 있습니다. IPOP dataset에서 추가로 제공되는 phosphor_20221211_Property_AF.csv 파일을 활용하면, 이미 생성되어있는 원자기반 feature들을 활용하여 PL&PLE peak positions, CIE(X,Y) coordinate, Quantum efficiency, PL decay time 값들을 빠르게 예측해볼 수 있습니다. 인공지능 플랫폼에서 제공되는 다양한 알고리즘을 적용해보고 모델 간 예측성능을 비교해봅니다.
----	-----------------	--

8	전이 금속 화합물의 양자 물성 예측 머신 학습	자연계에는 많은 수의 전이 금속이 존재하며, 이들 전이 금속이 들어있는 다양한 화합물 들은 생화학, 유기화학, 재료화학 등에서 매우 중요한 역할을 하고 있다. 본 프로젝트의 목표는 최근 공개된 대규모의 전이 금속 화합물 양자 계산 데이터베이스를 이용해서 전이 금속 화합물의 물리적 특성을 정확하게 예측하는 딥러닝 모델을 구축하는 것이다. 본 프로젝트에서 사용할 데이터셋은 the transition metal quantum mechanics (tmQM) data set, 줄여서 tmQM 데이터셋이다. 본 데이터셋은 약 8만 6천개의 전이금속 화합물의 DFT 계산을 이용해서 양자화학적 물성들, 전자 상태 에너지, HOMO-LUMO 에너지 차이, 쌍극자 모멘트 등,을 미리 계산해놓은 데이터셋이다. 수강생들은 본 데이터셋과 수업 시간에 배운 다양한 딥러닝 방법을 이용해서 전이 금속 화합물의 양자 물성을 예측하는 모델을 학습시켜 보고, 각 모델마다 어떤 특징을 가지는지 논의해 봄으로써, 다양한 머신러닝 방법이 화학과 어떻게 결합될 수 있는지 더 깊이 이해할 수 있을 것이다.(※ 데이터 제공 예정)
9	결합 생성 에너지 계산	제일원리 DFT 계산을 수행해서 반도체 안에서 생성되는 결함의 형성 에너지를 계산합니다. ASE를 이용해서 구조를 만들고, GPAW를 이용해서 DFT계산을 수행합니다. 제공된 주피터노트북을 활용하여 계산을 수행합니다. 여러 결함 가운데 원자 하나가 없어진 경우(vacancy)를 계산해봅시다. 각자 다른 교환-상관 퍼텐셜을 이용해서 계산하거나, 물질을 다이아몬드(C)로 바꿔볼 수 있습니다. (※ 주피터 노트북 파일 제공 예정)
10	열전 성능 예측 모델 생성	https://www.nature.com/articles/s41524-022-00897-2 에서 무기 열전 소재 데이터셋을 다운로드 받습니다. 교육 시간에 배우는 화학소재 인공지능 플랫폼, ChemAI (www.ai.chemdx.org)를 사용하여 여러 열전 물성들을 예측해봅니다. Atomic Formula와 Temperature를 사용하여 seebeck 계수, 전기전도도, 열전도도, power_factor, ZT 값들을 예측해 봅니다. ChemAI에서 제공하는 다양한 머신러닝 예측 알고리즘을 사용하여 예측을 수행하고 결과 값을 비교합니다. 최종적으로, 구축된 최적의 예측 모델을 Pearson's Crystal Database의 90,000 개의 무기소재 화학식에 적용해보고 결과 값을 분석해봅니다. Pearson's Crystal Database 는 https://pubs.acs.org/doi/full/10.1021/acs.jpcclett.8b00124 논문의 supporting information에서 Predicted band gaps of compounds from Pearson's Crystal Database (XLSX) 다운로드 받아 사용합니다.
11	AI를 활용한 격자 구조의 다중 물성 예측	나노 다공성 격자 구조의 일종인 금속유기골격체에서는 기공의 부피와 표면적을 통해 물질의 기체 운반자로서의 성능이나, 촉매로서의 성능을 알 수 있는 지표가 된다. 본 프로젝트에서는 mof-net 모델을 통해 두 가지 물성 (기공 부피, 표면적)을 각기 따로 예측하여보고 또, 전이 학습(transfer-learning)을 통해 연관지어 예측하여 본 뒤에 방법론에 따른 예측 성능의 차이를 확인하는 것을 목표로 한다. (※데이터 제공 예정)

소재정보학 개론

소재정보학이란

데이터가 이끄는 새로운 연구 패러다임

조성 · 구조 · 공정 · 물성의 관계를 데이터로 학습해, 원하는 소재를 직접 설계한다

2011 미국 소재 게놈 이니셔티브(MGI)

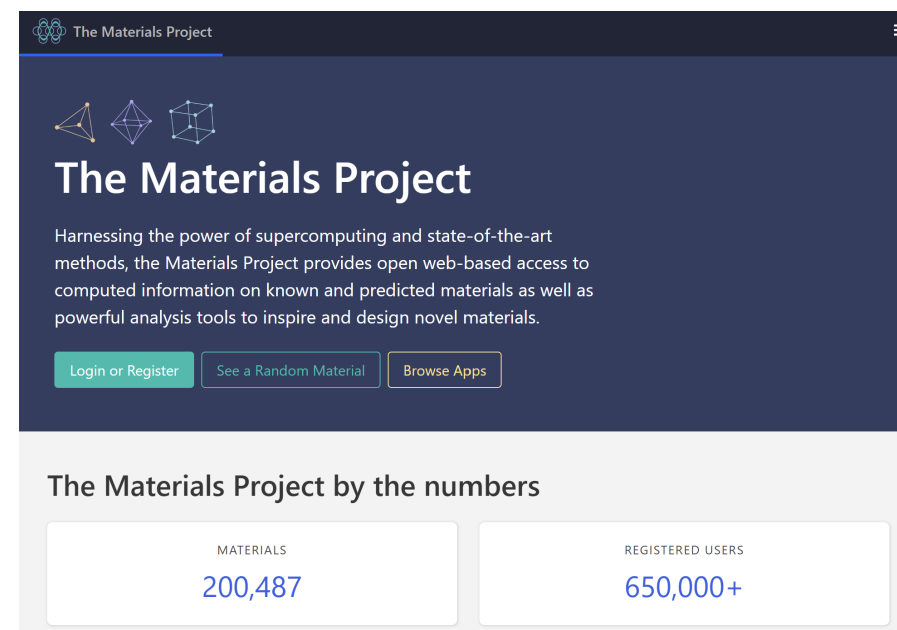
"개발 기간과 비용을 절반으로"
대규모 계산 데이터베이스 구축 시작

2014-16 머신러닝의 첫 성과

소재 물성 예측에서
의미 있는 정확도가 나오기 시작

2020s 딥러닝 · 생성모델 · LLM

구조를 직접 만들어 내고
실험까지 설계하는 단계로



Materials Project (materialsproject.org)

왜 지금인가

오래된 문제, 새로 갖추진 조건

지금까지의 소재 개발

- 실험실에서 시장까지 10~20년
- 연구자의 직관과 시행착오에 의존
- 실패한 실험은 기록조차 남지 않음

데이터

수백만 건의
공개 계산 데이터

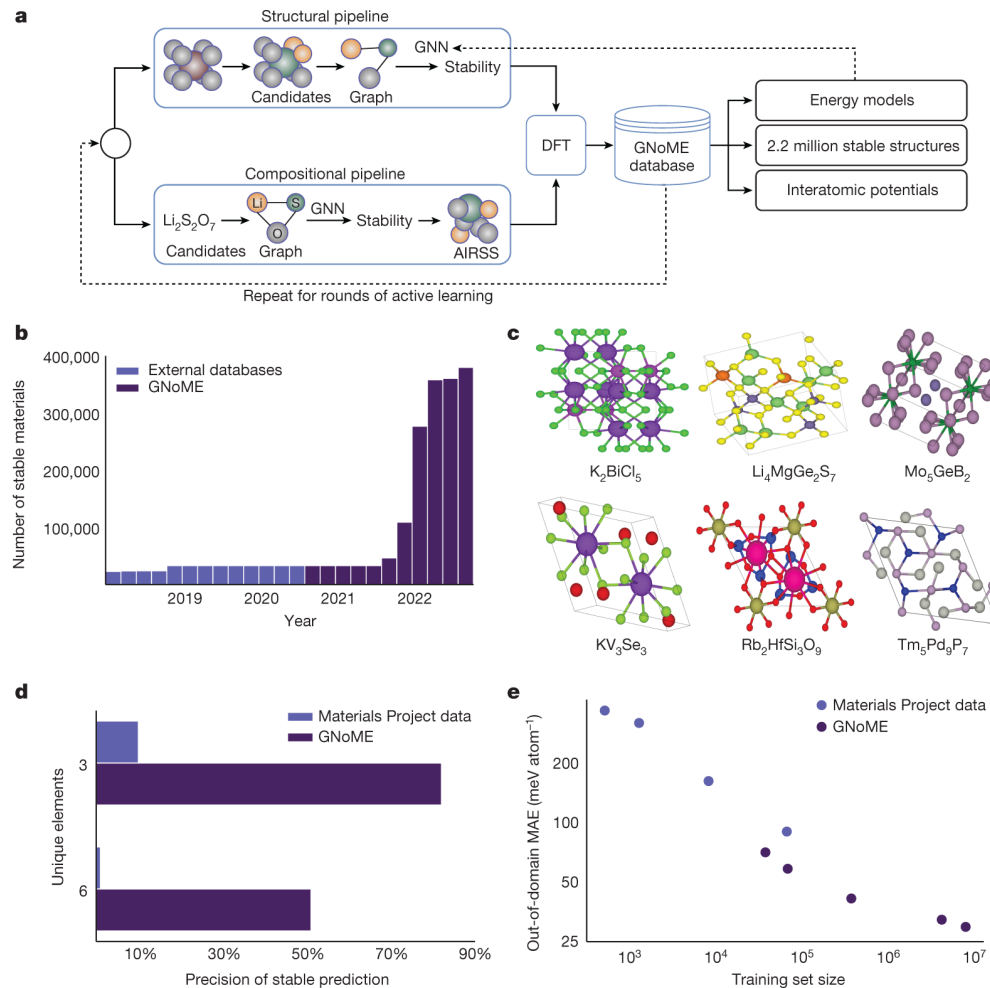
AI 모델

구조를 직접 다루는
딥러닝 모델

자동화

실험을 반복하는
로봇 장비

최근 모든 조건이 갖추어진 지금, 폭발적으로 연구가 이뤄짐



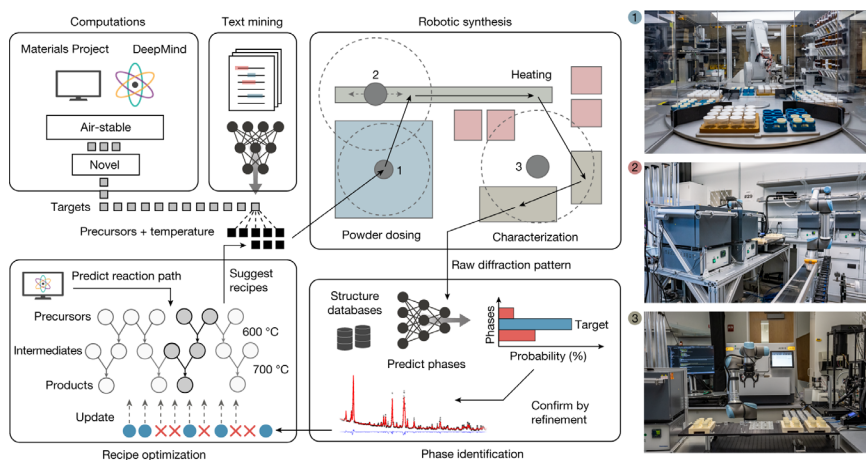
안정한 새로운 물질 탐색 GNoME Fig. 1 · Nature 624, 80 (2023)

소재정보학의 세가지 필수 요소

데이터 → 모델 → 실험 → 다시 데이터로 돌아오는 폐루프(Closed-loop)



실험 결과가 다시 데이터가 되어 모델을 개선한다



A-Lab Fig. 1 · Nature 624, 86 (2023) · CC BY 4.0

데이터베이스

믿을 수 있는 데이터를 어떻게 모을 것인가

AI 모델

적은 데이터로 어떻게 일반화할 것인가

자율실험실

예측을 얼마나 빨리 검증할 것인가

소재 데이터베이스

계산 데이터부터 표준화되어 대규모로 쌓이기 시작

데이터베이스	성격	규모
Materials Project	DFT 계산	20만 건 이상
OQMD	DFT 계산	약 140만 건
Alexandria / OMat24	DFT 계산	1.1억 건 계산
ICSD · CSD	실험 결정구조	수십만 건 (유상)
K-MDS (국내)	실험 + 계산	2027년 36만 건 목표

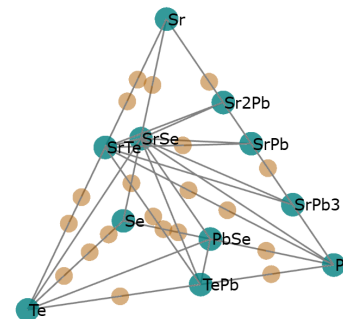
! 부족한 실험 데이터

공정 조건과 실패 기록이 거의 없음
기관마다 단위와 조건이 달라 그대로 합칠 수도 없다.



Home Materials Analysis Documentation API Download

The OQMD is a database of DFT calculated thermodynamic and structural properties of 1,407,395 materials, created in Chris Wolverton's group at Northwestern University.



Shortcuts

Search

Material Compositions

Query

Materials Data

Create

Phase Diagrams

Determine

Ground State Compositions (GCLP)

Visualize

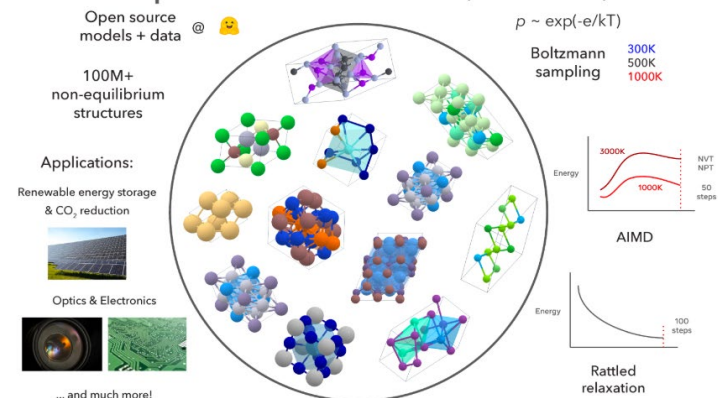
Crystal Structures

RESTful API

(OPTIMADE)

Meta Open Materials 2024 (OMat24) Dataset

Open Materials 2024 (OMat24)



AI 모델 ① 물성 예측

구조를 넣으면 물성이 나오는 모델 만들기

수작업 기술자

사람이 고른 특징값

랜덤포레스트 · XGBoost

그래프 신경망

구조를 직접 학습

CGCNN · M3GNet · MACE

범용 원자간 퍼텐셜

주기율표 전체를 하나로

CHGNet · MatterSim · UMA

- 이제는 범용 모델(foundation model)
- 한 번 크게 학습하고, 과제마다 미세조정
- 며칠 걸리던 계산이 초 단위로

! 벤치마크 점수가 전부는 아님

학습 범위를 벗어나면 성능이 급격히 떨어짐
표면과 결함 구조는 여전히 취약함

Matbench Discovery

Column presets: **Discovery** Geo Opt Phonons MD Diatomics

Discovery test set: Full Test Set **Unique Prototypes**

#	Model	CPS ↑	Acc	F1	DAF	Prec	MAE	R ²	KSRME	RMSD	CMD5	CDS	Params	Targets	Date Added	Links	r _{cut}	Training Size
1	TECE-OAM-RRA-1.0	0.908	0.978	0.929	6.073	0.928	0.018	0.871	0.093	0.058	0.630	0.270	222M	EFSG	2026-07-05	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
2	EquiFlashV2	0.907	0.978	0.929	6.069	0.928	0.018	0.873	0.094	0.058	n/a	n/a	44.9M	EFSG	2026-06-11	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
3	EquiformerV3+DeNS-OAM	0.902	0.978	0.931	6.074	0.928	0.018	0.868	0.118	0.059	n/a	n/a	30.3M	EFSG	2026-04-07	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
4	GRACE-3L-OAM-L	0.900	0.977	0.925	6.041	0.923	0.018	0.875	0.121	0.058	0.739	0.699	42.1M	EFSG	2026-07-02	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
5	PET-OAM-XL	0.898	0.977	0.924	6.075	0.929	0.019	0.864	0.119	0.060	0.634	0.388	730M	EFSG	2026-01-10	🔗 📄 📊	n/a	6.6M (113M) MPTrj+OMat24+sAl
6	TACE-OAM-L	0.889	0.972	0.910	5.898	0.902	0.020	0.868	0.126	0.061	0.728	0.761	82.9M	EFSG	2026-04-09	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
7	eSEN-30M-OAM	0.888	0.977	0.925	6.069	0.928	0.018	0.866	0.17	0.061	0.625	0.648	30.2M	EFSG	2025-03-17	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
8	EquiFlash	0.888	0.975	0.919	5.983	0.915	0.019	0.871	0.158	0.060	n/a	n/a	28.7M	EFSG	2025-06-23	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
9	Nequip-OAM-XL	0.886	0.971	0.906	5.869	0.897	0.020	0.872	0.125	0.063	0.623	0.796	32.1M	EFSG	2025-11-30	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
10	MatRIS-10M-OAM	0.877	0.976	0.921	6.039	0.923	0.019	0.871	0.218	0.060	0.581	0.678	10.4M	EFSGM	2025-10-29	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
11	SevenNet-Omni-i12*	0.873	0.971	0.906	5.954	0.910	0.021	0.868	0.192	0.062	0.626	0.696	54.9M	EFSG	2026-01-12	🔗 📄 📊	6 Å	243M COSMOSDatas
12	Nequip-OAM-L	0.870	0.967	0.893	5.823	0.890	0.022	0.865	0.166	0.065	0.670	0.818	9.6M	EFSG	2025-09-08	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
13	GRACE-2L-OAM-L	0.865	0.964	0.883	5.840	0.893	0.022	0.862	0.169	0.064	0.759	0.694	26.4M	EFSG	2025-09-09	🔗 📄 📊	6 Å	6.6M (113M) MPTrj+OMat24+sAl
14	ORB v3	0.860	0.971	0.905	5.912	0.904	0.024	0.821	0.21	0.075	n/a	n/a	25.5M	EFSG	2025-04-05	🔗 📄 📊	6 Å	6.47M (133M) MPTrj+Alex+OMat
15	DPA-4.0.1-Pro-MPTrj	0.840	0.956	0.857	5.609	0.857	0.029	0.836	0.211	0.069	0.598	n/a	22.8M	EFSG	2026-06-11	🔗 📄 📊	6 Å	146k (1.58M) MP

Matbench Discovery 리더보드 (2026년 기준)

AI 모델 ② 역설계

"이 구조의 물성은?" → "이 물성을 내는 구조를 만들어 줘"

순방향
구조 → 물성

역방향
물성 → 구조

GNoME

Nature 624, 80 (2023)

220만 개 구조를 탐색해
38만 개를 새 안정 물질로 판정

MatterGen

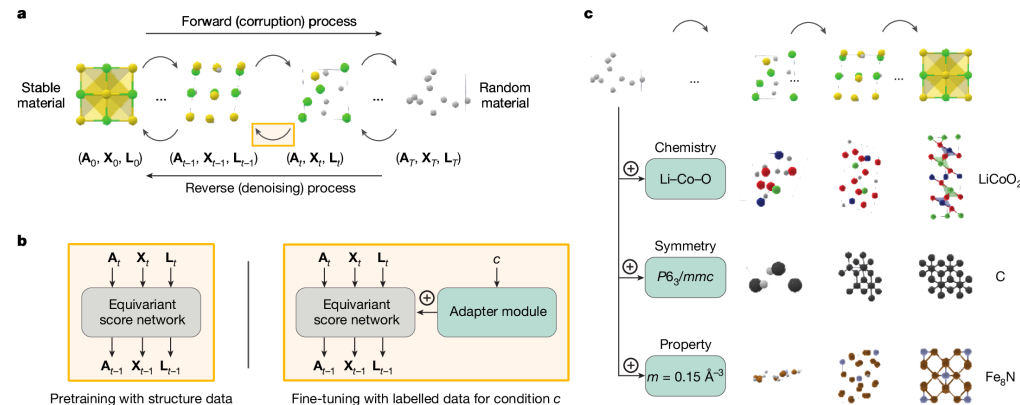
Nature 639, 624 (2025)

목표 물성을 주면
결정 구조를 직접 생성

LLM 에이전트

Nature 624, 570 (2023)

논문에서 합성 조건을 뽑고
실험 계획까지 작성



MatterGen Fig. 1 · Nature 639, 624 (2025) · CC BY 4.0

! 예측의 한계점

계산상 안정하다고 만들 수 있는 것은 아니다.
MatterGen이 실제로 합성까지 확인한 물질은
지금까지 단 하나(TaCr_2O_6)뿐. (bulk modulus
200GPa 이상)

자율실험실

AI로 실험 설계 → 로봇이 수행

1 실험 설계

능동학습이 조건 선택

2 로봇 합성

분주 · 혼합 · 열처리

3 자동 측정

XRD · UV-Vis · J-V

4 모델 갱신

결과를 학습에 반영

① → ② → ③ → ④ → 다시 ① 로, 사람 없이 하루 수십 회 반복

A-Lab

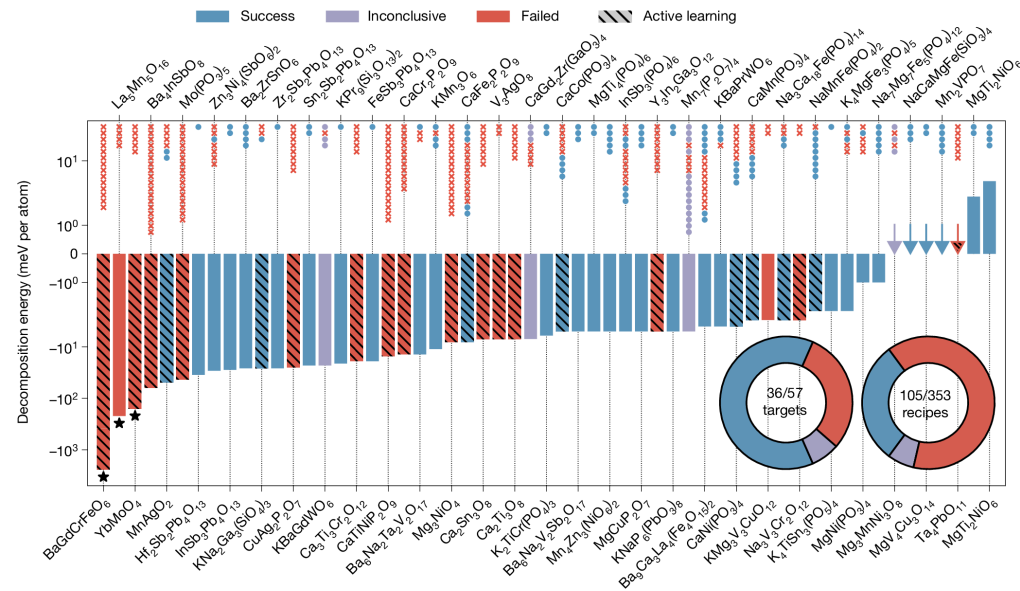
17일 만에 표적 57개 중 36개 자동 합성 Nature 624, 86 (2023)

흐름 기반 자율실험

같은 시간에 데이터 10배 확보 Nat. Chem. Eng. (2025)

SDL 2.0

장비끼리 연결되는 플랫폼으로 Mater. Horiz. (2026)



A-Lab Fig. 2 · Nature 624, 86 (2023) · CC BY 4.0

처음 발표: 41/58 합성 성공

2026년 1월 정정 36/57(63%) 합성 성공 (검증 중요!)

아직 풀리지 않은 문제

1 데이터 편중

무기결정에 쏠려 있고
고분자 · 비정질은 거의 없음

2 실패 기록 부재

논문은 성공만 심지만
모델에는 실패가 더 중요

3 합성 가능성

계산상 안정해도
만들 수 있는지는 별개

4 신규성 논쟁

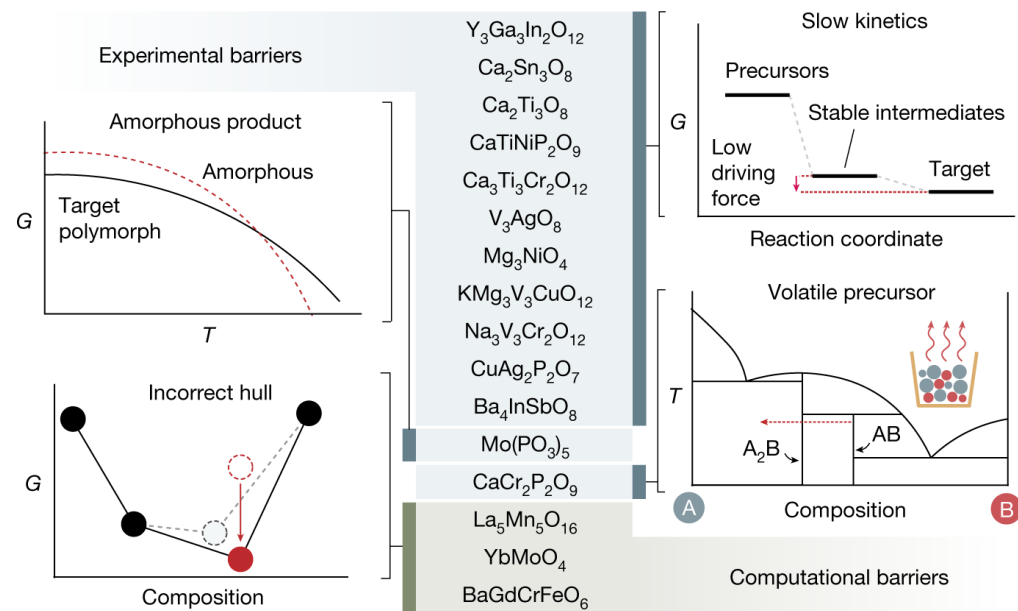
예측된 '신물질'이 정말
새로운지에 대한 반론

5 재현성

기관마다 측정 조건이 달라
데이터를 합칠 수 없음

6 해석 가능성

왜 그렇게 예측했는지
설명하지 못함



A-Lab Fig. 4 · Nature 624, 86 (2023) · CC BY 4.0

✓ **실패를 그림으로 남긴 드문 사례**
합성에 실패한 17개를 원인별로 분류

정리

소재정보학은 도구가 아니라 데이터가 순환하는 연구 방식

1 유기적으로 연결된 세가지 요소들

데이터베이스 · AI 모델 · 자율실험실 함께 운영이 필수

2 이제는 실험 검증이 병목

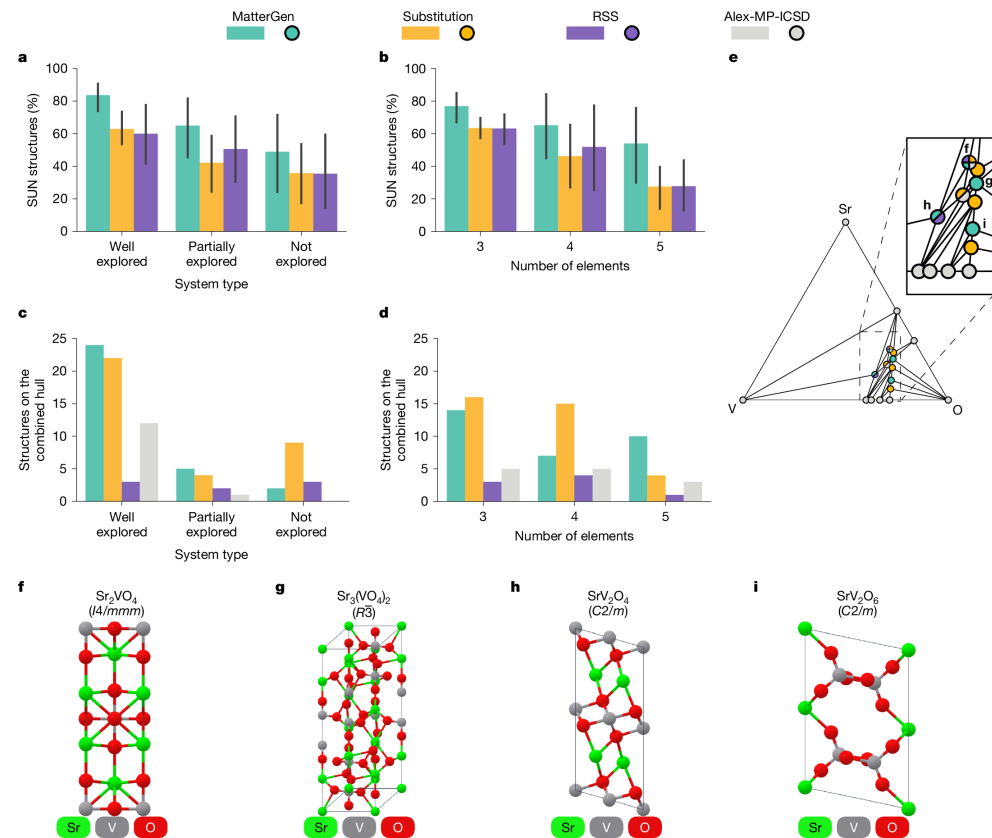
계산이 빨라진만큼 이제 검증 속도가 전체를 결정함

3 실제 실험 데이터의 중요성

실험실에서의 측정값과 실패 기록이 이 분야에서 가장 부족한 자원

더 읽어 볼 것

Lookman, Adv. Mater. (2026) · Tom, Chem. Rev. (2024) · Merchant, Nature 624, 80 (2023)



MatterGen Fig. 3 · Nature 639, 624 (2025) · CC BY 4.0

실습 환경 구축

(개인 노트북)

소재정보학 연구 워크플로우

파이썬은 이 다섯 단계를 하나로 잇는 도구



이 모든 단계를 하나의 언어로

- 엑셀 정리, MATLAB 계산, Origin 작도를 파이썬 스크립트 하나로 통합 가능
- 코드로 남기면 6개월 뒤에도, 다른 사람도 동일하게 재현 가능 — 연구 재현성의 핵심
- VASP·ORCA 등 계산 프로그램의 입력 생성과 결과 파싱도 대부분 파이썬으로 자동화

소재 분야에서 쓰는 파이썬 라이브러리

오늘은 왼쪽 두 칸만 다루고, 나머지는 학기 중 순차 진행

기초 · 데이터

numpy — 배열·행렬 연산
pandas — 표 형태 데이터
matplotlib — 그래프
seaborn — 통계 시각화

머신러닝

scikit-learn — 회귀·분류
xgboost / lightgbm — 부스팅
optuna — 하이퍼파라미터 탐색
shap — 모델 해석

딥러닝

pytorch — 표준 딥러닝 도구
torch-geometric — 그래프 신경망
mace / nequip — 원자 퍼텐셜
transformers — 언어모델

소재 · 화학 특화

pymatgen — 결정구조·상평형
ASE — 원자 시뮬레이션 인터페이스
RDKit — 분자 구조·기술자
matminer — 소재 기술자 생성

i 라이브러리 암기 불필요

필요할 때 검색해 문서를 읽는 능력이 더 중요. 오늘은 numpy · pandas · matplotlib 세 개만 익히면 충분

파이썬을 쓰려면 무엇이 필요한가

‘파이썬 설치’는 사실 세 가지를 준비하는 일

1

파이썬 본체 (인터프리터)

python.org에서 받는 프로그램 자체.
작성한 코드를 한 줄씩 실행하는 엔진

python 3.12

2

패키지 관리자

numpy, pandas 같은 남이 만든 코드를
찾아서 설치·업데이트해 주는 도구.

conda / pip

3

편집기 (에디터)

코드를 쓰고, 실행하고, 결과를 눈으로
확인하는 작업 공간.

Jupyter / VS Code

i

설치 프로그램 하나로 세 가지를 한 번에

Miniforge · Anaconda 같은 ‘배포판(distribution)’은 파이썬 본체와 패키지 관리자를 한 번에 설치
따라서 초보자는 python.org에서 파이썬을 따로 받는 것보다 배포판 사용이 유리
다만 배포판에 따라 라이선스 문제 발생 가능 — 다음 두 장 확인 후 결정

환경 구축 선택지 네 가지

설치 용량 · 라이선스 · 속도에서 차이 발생

	Miniforge	Anaconda Distribution	uv + pip	Google Colab
설치 용량	약 100 MB	약 5 GB	약 50 MB	설치 불필요
기본 패키지 채널	conda-forge (커뮤니티)	defaults (Anaconda사)	PyPI	구글 제공
라이선스 부담	없음	기관·규모에 따라 유상	없음	없음
설치 속도	빠름	느림	매우 빠름	—
GPU 계산	가능	가능	가능	무료 GPU 제공
이 수업 권장도	★ 표준	조건부	익숙해지면	보조 수단



이 수업의 표준은 Miniforge

가볍고 라이선스 부담 없음. conda 명령어는 Anaconda와 동일해 다른 환경에서도 그대로 사용 가능

우리 수업의 표준 조합

Miniforge + conda-forge + 가상환경

Miniforge

conda 명령어를 그대로 사용하면서
용량은 100 MB 남짓.
Anaconda 저장소를 쓰지 않아
라이선스 부담 없음

conda-forge

전 세계 자원봉사자가 관리하는
오픈 패키지 저장소.
완전 무료이며
패키지 종류가 가장 많음

가상환경

프로젝트마다 독립된 공간을 만들어
패키지 버전 충돌 방지.
망가지면 지우고 다시 만들 수 있는
안전장치

오늘 실습에서 만들 환경

환경 이름 : `matinfo` 파이썬 : 3.12
설치 패키지 : `numpy` · `pandas` · `matplotlib` · `scikit-learn` · `jupyterlab`
추가 예정 : `pytorch` (5주차) · `pymatgen`, `RDKit` (8주차)

Miniforge 내려받기

설치 파일은 GitHub 공식 저장소에서 내려받기

다운로드 주소 <https://conda-forge.org/download/>

내 컴퓨터	받아야 할 파일	비고
윈도우 (대부분)	Miniforge3-Windows-x86_64.exe	인텔·AMD CPU 노트북
맥 (M1~M4 칩)	Miniforge3-MacOSX-arm64.sh	2020년 이후 애플 실리콘
맥 (인텔)	Miniforge3-MacOSX-x86_64.sh	2020년 이전 모델
리눅스 / 서버	Miniforge3-Linux-x86_64.sh	연구실 계산 서버도 동일

i 내 윈도우가 64비트인지 확인하는 법

설정 → 시스템 → 정보 → 시스템 종류에서 '64비트 운영 체제' 확인. 2015년 이후 노트북은 거의 전부 64비트

Miniforge 설치하기 (윈도우)

여섯 단계, 약 5분 소요

1 내려받은 .exe 실행

'Windows에서 PC를 보호했습니다' 창이 뜨면 [추가 정보] → [실행] 선택

3 Just Me 선택

All Users 선택 시 관리자 권한이 필요하고 이후 권한 오류 발생 가능

5 체크박스 두 개

'Create shortcuts' 켜기 / 'Add to PATH'는 꺼 두기(권장) / 'Register as default Python' 켜기

✓ PATH 추가는 꺼 두는 편이 안전

'Miniforge Prompt'로 실행하면 문제 없음. 오히려 기존 파이썬과 충돌하지 않음

2 Next → I Agree

라이선스는 BSD-3 오픈소스. 동의 후 진행

4 설치 경로 확인

경로에 한글·공백이 있으면 반드시 변경. 예: C:\Wminiforge3

6 Install → Finish

설치 완료 후 시작 메뉴에 Miniforge Prompt 생성

맥 · 리눅스 설치와 설치 확인

터미널에서 스크립트를 실행하는 방식

맥 / 리눅스 설치

```
# 다운로드 폴더로 이동
$ cd ~/Downloads

# 설치 스크립트 실행 (파일명은 받은 것에 맞게)
$ bash Miniforge3-MacOSX-arm64.sh

# 엔터 → 약관 스페이스로 넘기고 yes
# 설치 경로 엔터 → 마지막 질문에 yes

# 터미널 종료 후 재실행
```

i 윈도우는 Miniforge Prompt에서

시작 메뉴에서 'Miniforge Prompt' 검색 후 실행. 일반 명령 프롬프트(cmd)에서는 conda 명령이 동작하지 않을 수 있음

설치가 잘 됐는지 확인

```
$ conda --version
conda 25.7.0

$ python --version
Python 3.12.9

$ conda config --show channels
channels:
  - conda-forge
```



채널에 conda-forge만 보이면 정상

defaults가 섞여 있으면 Anaconda 저장소 사용 중이라는 뜻. 정리 방법은 다음 장에서 설명

이미 Anaconda를 쓰고 있다면

삭제 불필요 — 패키지를 받아 오는 경로만 변경

① 지금 어떤 채널을 쓰는지 확인

```
$ conda config --show channels
channels:
  - defaults      ← Anaconda 저장소
  - conda-forge
```

② conda-forge만 쓰도록 변경

```
$ conda config --remove channels defaults
$ conda config --add channels conda-forge
$ conda config --set channel_priority strict
```

③ 기억해 둘 것

- 채널을 바꿔도 기존 defaults에서 설치한 패키지는 그대로 잔존. 새 가상환경 생성 권장
- Anaconda Navigator(그래픽 프로그램)는 Anaconda 저장소와 연결되어 있으므로 라이선스가 우려되면 사용 자제
- 기관 차원의 판단이 필요한 사안. 소속 기관 전산 담당자나 계약 부서에 문의 권장

i 완전히 정리하고 싶다면

제어판 → 프로그램 제거에서 Anaconda 삭제 후 Miniforge 신규 설치가 가장 확실
삭제 전 `conda env export`로 기존 환경 목록 백업

가상환경이 왜 필요한가

패키지를 한 곳에 몰아 설치하면 결국 충돌 발생

가상환경 없이 쓰면

- A논문 코드는 numpy 1.x, B논문 코드는 numpy 2.x 요구 → 한 쪽은 반드시 오류
- 패키지 하나 업데이트했더니 어제까지 돌던 코드가 오늘 에러
- 환경이 꼬이면 원인을 못 찾아 결국 노트북을 밀고 재설치
- 환경을 그대로 재현하도록 전달할 방법 없음

가상환경을 쓰면

- 프로젝트마다 독립된 방 — 옆방 패키지 버전이 바뀌어도 내 방은 그대로
- 망가지면 그 환경만 지우고 3분 만에 다시 만들면 끝
- environment.yml 파일 하나로 다른 사람 컴퓨터에 같은 환경을 재현
- 논문 투고 시 "이 환경에서 계산했다"를 명확히 밝힐 수 있음

비유하자면

실험실 후드에 비유 가능. 시약을 하나의 후드에 몰아넣으면 서로 반응하고 오염되지만, 실험별로 후드를 나누면 상호 간섭 없음. 가상환경은 코드를 위한 후드에 해당. 한 번 만들어 두면 내부에서 무엇을 설치·삭제하든 다른 프로젝트에 영향 없음

가상환경 만들고 들어가기

이름은 matinfo, 파이썬 버전은 3.12로 생성

```
# 1) matinfo 라는 이름으로 파이썬 3.12 환경 생성
$ conda create -n matinfo python=3.12

# Proceed ([y]/n)? 이 나오면 y 입력 후 엔터

# 2) 만든 환경 안으로 들어가기 (활성화)
$ conda activate matinfo

# 3) 프롬프트 앞이 (base) → (matinfo) 로 바뀌면 성공
(matinfo) C:\Users\yeri>

# 4) 환경 목록 확인 - * 가 현재 위치
$ conda env list
```

왜 3.12인가

최신은 3.14이나, 소재·머신러닝 패키지는 보통 한두 버전 뒤를 따라옴

3.12는 주요 패키지가 모두 지원하면서 충분히 최신인, 현재 가장 안전한 선택

환경 이름 규칙

영문 소문자·숫자·하이픈만 사용. 한글이나 공백 포함 시 경로 오류 발생

프로젝트 이름을 그대로 사용하면 혼동 방지

! activate를 잊으면 생기는 일

(base)에서 설치한 패키지는 matinfo 환경에서 미인식. 터미널을 열 때마다 conda activate matinfo 실행 필요

conda와 pip, 어떻게 섞어 쓰나

둘 다 패키지 설치 도구이나 성격이 다름

	conda	pip
설치 대상	파이썬 패키지 + C/포트란 라이브러리 + CUDA 같은 시스템 의존성까지	파이썬 패키지만
저장소	conda-forge 등 채널	PyPI
의존성 처리	환경 전체를 함께 풀어 충돌을 미리 막음 (대신 속도가 느림)	하나씩 설치, 충돌 가능 (대신 빠름)
잘 맞는 경우	numpy, scipy, pymatgen, RDKit 등 컴파일이 필요한 과학 패키지	PyPI에만 있는 최신 패키지, PyTorch 공식 배포판

실무에서 쓰는 순서

- conda로 최대한 설치하고, conda에 없는 것만 pip으로 보충
- 순서를 바꾸면(pip 먼저 → conda 나중) conda가 pip 설치 파일을 덮어써 환경 손상 발생
- pip 설치 시 반드시 해당 가상환경을 activate한 상태여야 함

수업용 패키지 설치하기

(matinfo) 환경에 진입한 상태에서 실행

```
# 데이터 분석 기본 4종 + 주피터
$ conda install numpy pandas matplotlib scikit-learn jupyterlab

# 여러 개를 한 줄에 쓰면 의존성을 한 번에 해결

# 나중에 추가로 자주 쓰게 될 것들
$ conda install seaborn openpyxl tqdm ipywidgets

# 설치된 목록 확인
$ conda list

# 특정 패키지만 검색
$ conda list numpy
```

설치가 오래 걸린다면

conda가 의존성을 해석하는 중. 'Solving environment'에서 수 분간 멈춘 듯 보여도 정상

openpyxl은 왜?

pandas로 엑셀(.xlsx) 파일을 읽으려면 이 패키지 필요

실험 데이터는 대부분 엑셀 형식이므로 미리 설치

✓ 환경을 통째로 저장하고 복원하기

conda env export > environment.yml 로 현재 환경을 파일 하나에 기록 가능

다른 컴퓨터에서 conda env create -f environment.yml 실행 시 동일 환경 재현. 논문 보충자료용으로 적합

PyTorch는 pip으로 설치

conda로 만든 가상환경 안에서 pip 사용이 공식 안내

```
# 반드시 환경을 활성화한 상태에서
$ conda activate matinfo

# CPU만 있는 노트북
$ pip install torch torchvision

# NVIDIA GPU가 있는 경우 (예: CUDA 12.x)
$ pip install torch --index-url https://download.pytorch.org/whl/cu126
```

- PyTorch 공식 배포판은 conda 채널 미지원
- 환경은 conda로 만들고 설치만 pip으로 수행. 두 도구를 혼용하는 것이 아님
- 설치 명령은 버전.CUDA에 따라 상이. pytorch.org/get-started 에서 사양 선택 후 나오는 명령을 그대로 복사

PyTorch는 5주차에 함께 설치

용량이 크고 CUDA 버전 확인이 필요해 금일 설치 제외. GPU 보유자는 사전에 `nvidia-smi` 실행 후 버전 기록

설치가 제대로 됐는지 확인

아래 코드 실행 시 오류가 없으면 환경 구축 완료

```
# check_env.py
import sys
import numpy as np
import pandas as pd
import matplotlib
import sklearn
print("Python      :", sys.version.split()[0])
print("numpy       :", np.__version__)
print("pandas       :", pd.__version__)
print("matplotlib   :", matplotlib.__version__)
print("sklearn      :", sklearn.__version__)
print("환경 구축 완료!")
```

! ModuleNotFoundError가 난다면

- ① conda activate matinfo 를 했는지 확인
- ② where python 으로 실행 중인 파이썬 경로 확인
- ③ 경로에 miniforge3\envs\matinfo 가 있어야 정상

실행 결과 예시

```
$ python check_env.py
Python      : 3.12.9
numpy       : 2.2.3
pandas      : 2.2.3
matplotlib  : 3.10.0
sklearn     : 1.6.1
환경 구축 완료!
```

✓ 버전 숫자는 달라도 무방

설치 시점에 따라 상이. ModuleNotFoundError 없이 끝까지 출력되는 것이 중요

conda 명령어

자주 활용하는 콘다 명령어

환경 관리

```
conda env list
```

환경 목록 보기

```
conda create -n 이름 python=3.12
```

새 환경 만들기

```
conda activate 이름
```

환경 들어가기

```
conda deactivate
```

환경 빠져나오기

```
conda env remove -n 이름
```

환경 통째로 삭제

패키지 관리

```
conda install 패키지
```

설치

```
conda install 패키지=1.2.3
```

버전 지정 설치

```
conda list
```

설치 목록 보기

```
conda update 패키지
```

업데이트

```
conda remove 패키지
```

삭제

공유 · 백업

```
conda env export > env.yml
```

현재 환경 파일로 저장

```
conda env create -f env.yml
```

파일로 환경 재현

```
conda clean --all
```

캐시 정리(용량 확보)

```
conda config --show channels
```

채널 확인

```
conda --version
```

conda 버전 확인

명령이 기억나지 않으면 conda 뒤에 --help 추가 시 설명 출력. 예: conda create --help

설치할 때 자주 만나는 오류 다섯 가지

대부분 이 범위 안에서 해결 가능

`'conda' is not recognized`

PATH 미등록 상태. 윈도우는 시작 메뉴의 Miniforge Prompt로 실행

`Solving environment: failed`

요청한 패키지 간 호환 불가. 버전 지정을 제거하거나 새 환경에서 재설치

`PackagesNotFoundError`

해당 패키지가 conda-forge에 없는 경우. pip install 패키지 로 설치

`Permission denied / 접근 거부`

관리자 권한 문제. 설치 시 All Users 대신 Just Me 선택. 백신 차단 사례도 있음

`ModuleNotFoundError`

환경 미활성화 또는 다른 파이썬 실행 중. where python으로 경로 확인

코드를 실행하는 세 가지 방식

같은 파이썬이지만 작업 방식에 차이 있음

JupyterLab

브라우저에서 셀 단위 실행

- 한 줄씩 실행하며 결과 확인
- 그래프가 바로 아래에 표시
- 데이터 탐색·분석에 최적
- 긴 프로그램 관리에는 불편

VS Code

본격적인 코드 편집기

- .py 스크립트 작성에 강함
- 자동완성·디버깅·깃 연동
- 주피터 노트북도 열 수 있음
- 처음엔 설정할 것이 조금 많음

Google Colab

설치 없이 웹에서

- 브라우저만 있으면 바로 시작
- 무료 GPU 사용 가능
- 공유·협업이 쉬움
- 내 컴퓨터 파일 접근은 번거로움



오늘 실습은 JupyterLab으로 진행

결과를 즉시 확인할 수 있어 학습 초기에 적합.

학기 중반 이후 코드가 길어지면 VS Code로 전환

JupyterLab 실행하기

터미널에서 한 줄 입력이면 브라우저 실행

```
# 1) 작업할 폴더를 하나 만들고 이동
$ cd C:\Users\yeri\matinfo-2026

# 2) 환경 활성화 (안 했다면)
$ conda activate matinfo

# 3) 주피터랩 실행
$ jupyter lab

→ 브라우저 자동 실행
→ 안 열리면 터미널에 뜬 http://localhost:8888/...
   주소를 복사해 브라우저에 입력
```

- 새 노트북 만들기
- 왼쪽 파일 탐색기에서 Launcher → Notebook → Python 3 클릭
- 파일 확장자
- 노트북은 .ipynb로 저장. 코드와 결과, 설명이 한 파일에 통합
- 종료하는 법
- 브라우저 종료만으로는 미종료. 터미널에서 Ctrl+C 두 번 입력 필요

i 작업 폴더 사전 지정

jupyter lab 실행 위치가 곧 노트북의 최상위 폴더. 바탕화면이나 다운로드 폴더에서 실행하면 파일 관리가 어려워짐. matinfo-2026 등 수업용 폴더를 만들어 그 안에서만 작업

노트북 사용법 : 셀 단위로 생각하기

코드를 잘게 나누어 한 덩어리씩 실행하고 확인

셀의 두 가지 종류

Code 셀

파이썬 코드를 입력해 실행하면 바로 아래에 결과 출력

Markdown 셀

제목.설명.수식 작성용. 실행 시 서식이 적용된 글로 변환

실행 결과 표시 형태

```
In [1]: a = 3.2      # 밴드갭 (eV)
        b = 1240 / a # 파장 (nm)
        print(b)
```

Out: 387.5

꼭 알아야 할 개념

- 실행 순서가 곧 상태
- 위에서 아래 순이 아니라 실행한 순서대로 값이 남음. In [] 안의 번호가 실행 순서
- 이상하면 커널을 다시 시작
- Kernel → Restart Kernel and Run All Cells 실행 시 처음부터 재실행
- 제출 전에는 항상 전체 재실행
- '내 화면에서는 되는데 다른 컴퓨터에서 안 되는' 사고의 90%가 여기서 발생

주피터 단축키

마우스 사용을 줄일수록 분석 속도 향상

실행 관련

Shift + Enter	실행하고 다음 셀로
Ctrl + Enter	실행하고 그 자리에
Alt + Enter	실행하고 아래에 새 셀
Esc	명령 모드로 (파란 테두리)
Enter	편집 모드로 (초록 테두리)

셀 편집 관련

a / b	위 / 아래에 셀 추가
d, d	셀 삭제 (d 두 번)
z	삭제 취소
m / y	마크다운 / 코드 셀로 변경
Shift + Tab	함수 설명 보기

✓ 도움이 필요할 때 쓰는 세 가지

- ① 함수 이름 뒤에 물음표 : np.mean? → 설명서 표시
- ② Tab 키 : 이름 앞부분 입력 후 Tab → 자동완성 목록 표시
- ③ Shift + Tab : 괄호 안에 커서를 두고 입력 → 필요한 인자 안내

VS Code 준비하기

학기 후반, 코드가 길어지면 전환하는 도구

1 VS Code 설치

code.visualstudio.com 에서 무료 배포

2 Python 확장 설치

왼쪽 확장 아이콘 → 'Python' 검색 → Microsoft 것 설치

3 Jupyter 확장 설치

동일한 방식으로 'Jupyter' 설치. .ipynb 파일을 VS Code에서 바로 열람 가능

4 인터프리터 선택

Ctrl+Shift+P → 'Python: Select Interpreter' → matinfo 환경 선택

④번이 가장 중요

- 인터프리터 미선택 시 VS Code가 다른 파이썬으로 실행해 '설치했는데 찾지 못하는' 상황 발생
- 화면 하단에 현재 선택된 환경 이름이 표시되므로 matinfo 여부 상시 확인
- AI 코드 도우미(GitHub Copilot 등) 사용 계획이 있으면 VS Code가 유리

Google Colab : 설치 없이 시작하기

노트북 설치가 끝내 되지 않는 경우의 대안

1 colab.research.google.com 접속

구글 계정 로그인만으로 사용 가능. 설치 불필요

2 새 노트북 만들기

[새 노트] 클릭 → 주피터와 거의 동일한 화면

3 패키지 설치는 느낌표로

셀 안에서 !pip install pymatgen 처럼 앞에 ! 추가

4 GPU 켜기

런타임 → 런타임 유형 변경 → 하드웨어 가속기에서 GPU 선택

구글 드라이브의 파일 읽기

```
from google.colab import drive
drive.mount('/content/drive')
```

이후 /content/drive/MyDrive/... 경로로 접근

! 알아 둘 제약

일정 시간 미사용 시 연결 종료 및 설치 패키지 소멸
무료 GPU는 사용량 제한 존재. 장시간 계산에는 부적합

언제 무엇을 쓸까

상황별 정리

이런 상황이라면	권장 도구	이유
데이터를 처음 열어 보고 훑을 때	JupyterLab	한 줄씩 실행하며 결과를 눈으로 확인
그래프를 여러 번 고쳐 그릴 때	JupyterLab	코드와 그림이 한 화면에
수백 줄짜리 분석 파이프라인	VS Code	파일 분리·디버깅·버전 관리
계산 서버에 코드를 올릴 때	VS Code + .py	터미널에서 바로 실행 가능
노트북에 설치가 도저히 안 될 때	Google Colab	설치 없이 즉시 시작
GPU가 필요한데 장비가 없을 때	Google Colab	무료 GPU 할당
팀원과 코드를 같이 볼 때	Colab 또는 GitHub	링크 공유·주석 달기

셋 다 동일한 파이썬 사용. 도구를 바꿔도 코드를 새로 배울 필요 없음

파이썬 코드의 형태

다른 언어를 해 봤다면 가장 먼저 눈에 띄는 두 가지

```
# 주석 (실행되지 않음)
sample = "MAPbI3"          # 시료 이름
bandgap = 1.55              # 밴드갭 (eV)

if bandgap < 1.7:
    print(sample, "는 단일접합에 적합")
    print("밴드갭:", bandgap, "eV")
else:
    print(sample, "는 상부셀 후보")

# 실행 결과
# MAPbI3 는 단일접합에 적합
# 밴드갭: 1.55 eV
```

! IndentationError를 만나면

들여쓰기 불일치가 원인. 에디터에서 '탭을 공백으로 변환' 설정 시 대부분 예방

① 중괄호 대신 들여쓰기

C나 Java의 { } 대신 들여쓰기로 코드 덩어리 구분
공백 4칸이 표준. 탭과 공백 혼용 시 오류 발생

② 세미콜론 없음

줄 끝에 ; 불필요. 줄바꿈이 곧 문장의 끝
변수의 자료형도 사전 선언 불필요

변수와 기본 자료형

값에 이름을 붙이는 것이 전부

```
# 변수 = 값 (선언 없이 바로 대입)
name      = "MAPbI3"      # str   문자열
bandgap   = 1.55           # float 실수
n_cells   = 12            # int    정수
is_stable = True          # bool   참/거짓
note      = None          # None   값 없음

# 자료형 확인
print(type(bandgap))      # <class 'float'>

# 형 변환
text = str(bandgap)       # '1.55'
num  = float("1.55")      # 1.55
cnt  = int("12")          # 12

# 여러 개 한 번에
a, b = 1.55, 2.30
```

핵심 정리

- 변수 이름 규칙
 - 영문·숫자·밑줄만 사용. 숫자로 시작 불가. 대소문자 구분
 - bandgap과 Bandgap은 서로 다른 변수
- 읽기 좋은 이름 사용
 - a, b, x1 보다 bandgap, pce, thickness_nm 이 적절. 6개월 뒤를 위한 배려
- None은 0이나 빈 문자열과 다름
- 미측정 값, 결측치 표현에 사용

숫자와 연산자

계산기로 쓰는 파이썬

```
a, b = 7, 2

print(a + b)    # 9
print(a - b)    # 5
print(a * b)    # 14
print(a / b)    # 3.5    ← 항상 실수
print(a // b)   # 3      ← 몫(버림)
print(a % b)    # 1      ← 나머지
print(a ** b)   # 49     ← 거듭제곱

# 소재 계산 예시 : 밴드갭 → 흡수 파장
bandgap = 1.55                # eV
wavelength = 1239.84 / bandgap
print(round(wavelength, 1))   # 799.9 nm

# 비교 연산 → True / False
print(bandgap < 1.7)         # True
```

핵심 정리

- / 와 // 구분 필요
- / 는 나눗셈 결과를 항상 실수로 반환. 정수 몫이 필요하다면 // 사용
- round() 는 반올림
- round(값, 소수점자리) 형태. 표 작성 시 빈번히 사용
- 실수 계산의 미세한 오차
- 0.1 + 0.2 가 0.30000000000000004 로 출력. 실수를 2진수로 저장하기 때문이며 파이썬만의 문제 아님
- 값 비교 시 == 대신 abs(a-b) < 1e-9 처럼 오차 범위로 판단

문자열 다루기

시료 이름, 파일 경로, 라벨 — 문자열 사용 빈도 높음

```
s = "CH3NH3PbI3"

print(len(s))          # 10 길이
print(s.upper())        # 대문자로
print(s.replace("I", "Br"))
print(s[0])            # 'C' 첫 글자
print(s[-1])           # '3' 마지막 글자
print(s[0:4])          # 'CH3N' 잘라내기

# 나누고 합치기
line = "MAPbI3, 1.55, 20.3"
parts = line.split(", ")
print(parts)           # ['MAPbI3', '1.55', '20.3']
print("-".join(parts)) # MAPbI3-1.55-20.3

# 앞뒤 공백 제거 (데이터 정제 필수)
print("  MAPbI3  ".strip())
```

핵심 정리

- 따옴표는 ' 나 " 아무거나
- 다만 한 파일 안에서는 하나로 통일하는 것이 바람직
- 인덱스는 0부터
- 첫 글자가 s[0]. 음수는 뒤에서부터 계산
- **split()**과 **strip()**은 꼭 기억
- 실험 데이터를 텍스트 파일에서 읽어 올 때 거의 매번 사용
- **문자열은 수정 불가**
- s[0] = 'X' 는 오류. replace처럼 새 문자열을 생성해 반환받아야 함

f-string : 문자열 안에 값 끼워 넣기

결과를 정돈해 출력할 때 가장 많이 사용

```
sample = "MAPbI3"
bandgap = 1.5487
pce      = 23.4567

# 문자열 앞에 f 를 붙이고 { } 안에 변수
print(f"{sample}의 밴드갭은 {bandgap} eV")
# MAPbI3의 밴드갭은 1.5487 eV

# 소수점 자리 지정 : {값:.2f}
print(f"PCE = {pce:.2f} %")
# PCE = 23.46 %

# 자리 맞춤 : {값:>8} 오른쪽 정렬 8칸
print(f"{sample:>10} {pce:>8.2f}")
#      MAPbI3      23.46

# 지수 표기 : {값:.3e}
print(f"{0.000123:.3e}") # 1.230e-04
```

왜 f-string인가

% 나 .format() 대신 f-string이 짧고 읽기 쉬움
현재 사실상 표준이므로 이것만 익히면 충분

자주 쓰는 서식

:.2f	소수점 둘째 자리
:.3e	지수 표기
>10	오른쪽 정렬 10칸
,	천 단위 쉼표
:.1%	백분율 표기

리스트 : 여러 값을 순서대로 담기

측정값 목록, 시료 목록 — 사용 빈도가 가장 높은 자료구조

```
gaps = [1.55, 1.62, 2.30, 1.48]

print(gaps[0])      # 1.55 첫 번째
print(gaps[-1])     # 1.48 마지막
print(gaps[1:3])    # [1.62, 2.3] 구간
print(len(gaps))    # 4 개수

# 값 추가·삽입·삭제
gaps.append(1.71)    # 끝에 추가
gaps.insert(0, 1.40) # 0번 자리에
gaps.remove(2.30)    # 값으로 삭제
last = gaps.pop()    # 마지막 꺼내기

# 정렬과 통계
gaps.sort()          # 오름차순 정렬
print(min(gaps), max(gaps), sum(gaps))
print(sum(gaps) / len(gaps)) # 평균
```

핵심 정리

- 대괄호 `[]` 로 생성
- 순서가 있고, 이후 값 변경 가능
- 슬라이싱 `[시작:끝]`
- 끝 번호는 미포함. `gaps[1:3]`은 1번과 2번만 해당
- `sort()`는 원본을 변경
- 원본 유지가 필요하다면 `sorted(gaps)` 사용
- 서로 다른 자료형도 섞을 수 있지만
- 실무에서는 같은 종류만 담는 것이 안전

튜플과 집합

리스트와 유사하나 쓰임새가 다름

튜플 () — 바꿀 수 없는 목록

```
abc = (12.4, 12.4, 6.35) # 격자상수
print(abc[0])           # 12.4

# abc[0] = 13.0 → TypeError

# 값 여러 개를 한 번에 풀기
a, b, c = abc

# 함수가 값 두 개를 돌려줄 때도 튜플
def minmax(xs):
    return min(xs), max(xs)
lo, hi = minmax([1.5, 2.3, 1.9])
```

- 생성 후 변경 불가하므로, 실수로 바뀌면 안 되는 값에 사용
- 좌표, 격자상수, 함수 반환값 등에 자주 등장

집합 { } — 중복 없는 모음

```
els = ["Pb", "I", "Pb", "C", "N", "I"]
uniq = set(els)
print(uniq)      # {'Pb', 'I', 'C', 'N'}
print(len(uniq)) # 4

a = {"Pb", "I", "Br"}
b = {"Sn", "I", "Br"}
print(a & b)      # 교집합 {'I', 'Br'}
print(a | b)      # 합집합
print(a - b)      # 차집합 {'Pb'}
```

- 중복 제거 시 set() 사용이 가장 빠름
- '두 데이터셋에 공통으로 등장하는 원소' 같은 계산에 유용

딕셔너리 : 이름표를 붙여 저장하기

실험 한 건의 정보를 통째로 담기에 적합

```
cell = {
    "sample": "MAPbI3",
    "bandgap": 1.55,
    "pce": 23.4,
    "date": "2026-08-31",
}

print(cell["pce"])          # 23.4
cell["voc"] = 1.12          # 항목 추가
cell["pce"] = 24.1          # 값 수정

# 없는 키를 안전하게 읽기
print(cell.get("jsc", 0))   # 없으면 0

# 전체 훑기
for key, value in cell.items():
    print(key, ":", value)
```

핵심 정리

- { 키 : 값 } 형태
- 순서 번호 대신 이름(키)으로 값 추출
- 키는 보통 문자열
- cell["pce"] 처럼 읽으면 무슨 값인지 코드만 봐도 확인 가능
- **get() 사용이 안전**
- 없는 키를 []로 읽으면 KeyError로 프로그램 중단
- **실험 데이터와 잘 맞음**
- 딕셔너리 여러 개를 리스트에 담으면 그대로 표(DataFrame)로 변환

조건문 : if / elif / else

데이터를 분류할 때 가장 먼저 쓰는 문법

```
bandgap = 1.55

if bandgap < 1.3:
    grade = "하부셀 후보"
elif bandgap < 1.8:
    grade = "단일접합 최적"
else:
    grade = "상부셀 후보"

print(grade)      # 단일접합 최적

# 조건 여러 개 묶기
pce, stable = 23.4, True
if pce > 20 and stable:
    print("보고 대상")

if not stable or pce < 15:
    print("재측정 필요")
```

핵심 정리

- 콜론(:)과 들여쓰기가 짝
- if 줄 끝에 : 를 쓰고, 다음 줄은 4칸 들여쓰기
- elif는 개수 제한 없음
- 위에서부터 순서대로 검사하고, 처음 참인 것 하나만 실행
- and · or · not
- &&, ||, ! 대신 영어 단어 사용. 읽기는 쉬우나 초기에는 혼동 가능
- == 와 = 는 다름
- = 는 대입, == 는 비교. 가장 흔한 초보 실수

반복문 : for / while

리스트를 하나씩 꺼내 처리하는 것이 기본형

```
gaps = [1.55, 1.62, 2.30, 1.48]

# 값을 하나씩 꺼내며 반복
for g in gaps:
    print(g, "eV")

# 정해진 횟수만큼
for i in range(5):          # 0,1,2,3,4
    print(i)

for i in range(1, 11, 2): # 1,3,5,7,9
    print(i)

# 조건이 참인 동안 반복
temp = 300
while temp < 400:
    temp += 25
print(temp)    # 400
```

핵심 정리

- **for 변수 in 목록:**
- 리스트·문자열·딕셔너리 등 순서가 있는 자료는 모두 반복 가능
- **range(끝) / range(시작, 끝, 간격)**
- 끝 값은 미포함. range(5)는 0부터 4까지
- **while은 조건이 거짓이 될 때까지**
- 값을 바꾸는 줄 누락 시 무한 반복. 중단은 Ctrl+C 또는 커널 중단
- 실무에서는 for 사용 비중이 압도적

반복문을 편하게 쓰는 도구들

enumerate · zip · break · continue

```
names = ["MAPbI3", "FAPbI3", "CsPbI3"]
gaps = [1.55, 1.48, 1.73]

# 번호와 값을 함께
for i, name in enumerate(names, start=1):
    print(i, name)
# 1 MAPbI3 / 2 FAPbI3 / 3 CsPbI3

# 두 리스트를 나란히
for name, g in zip(names, gaps):
    print(f"{name}: {g} eV")
```

enumerate와 zip을 쓰면

인덱스 변수를 별도로 만들거나 names[i], gaps[i] 처럼 번호로 접근할 필요 없음
코드가 짧아지고 실수도 감소

```
for g in [1.55, 1.62, 2.30, 1.48]:
    if g > 2.0:
        continue          # 이번 것만 건너뛰기
    if g < 1.5:
        break              # 반복 전체 중단
    print(g)
# 1.55 1.62 출력 후 종료

# 리스트 컴프리헨션 : 반복을 한 줄로
gaps = [1.55, 1.62, 2.30, 1.48]
low = [g for g in gaps if g < 1.7]
```

리스트 컴프리헨션

[식 for 변수 in 목록 if 조건] 형태로 새 리스트를 한 줄에 생성
파이썬 코드에서 자주 등장하므로 눈에 익혀 둘 것

함수 : 반복되는 작업에 이름 붙이기

같은 코드를 세 번 복사하고 있다면 함수로 전환할 시점

```
def gap_to_wavelength(bandgap):
    """밴드갭(eV)을 파장(nm)으로 변환"""
    return 1239.84 / bandgap

print(gap_to_wavelength(1.55))    # 799.9

# 기본값이 있는 인자
def classify(gap, low=1.3, high=1.8):
    if gap < low:
        return "하부셀"
    elif gap < high:
        return "단일접합"
    return "상부셀"

print(classify(1.55))              # 단일접합
print(classify(1.55, high=1.5))    # 상부셀
```

핵심 정리

- **def 이름(인자):**
- 함수를 정의하고, return으로 결과 반환
- **return이 없으면 None 반환**
- print만 하고 return을 생략하면 값을 받아 쓸 수 없음. 빈번한 실수
- **""" """ 안의 설명글 (docstring)**
- 함수의 기능을 기록. 주피터에서 함수명? 으로 확인 가능
- **기본값 인자**
- 자주 쓰는 값을 미리 지정하면 호출이 간결

모듈과 import

남이 만든 코드를 가져다 쓰는 방법

```
# 1) 모듈 전체 가져오기
import math
print(math.sqrt(2))      # 1.4142...
print(math.pi)          # 3.14159...

# 2) 짧은 이름을 붙여서 (관례)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 3) 필요한 것만 콧 집어서
from math import sqrt, log
print(sqrt(2))

# 4) 내가 만든 파일도 모듈
#     utils.py 안의 함수 쓰기
from utils import gap_to_wavelength
```

핵심 정리

- **as** 는 별명
- numpy를 np, pandas를 pd로 쓰는 것이 전 세계 공통 관례. 그 대로 사용
- **import**는 파일 맨 위에 모아서
- 코드 중간에 흩어져 있으면 의존 관계 파악이 어려움
- **from ... import *** 사용 금지
- 이름의 출처를 알 수 없어 이후 디버깅이 어려움
- **ModuleNotFoundError**가 나면
- 미설치 상태이거나 다른 환경에서 실행 중

파일 읽고 쓰기

실험 데이터는 결국 파일에서 유입

```
# 쓰기 : 'w' (덮어쓰기), 'a' (이어쓰기)
with open("result.txt", "w", encoding="utf-8") as f:
    f.write("sample,bandgap,pce\n")
    f.write("MAPbI3,1.55,23.4\n")

# 읽기 : 'r'
with open("result.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())

# 한 번에 전부 읽어 리스트로
with open("result.txt", encoding="utf-8") as f:
    lines = f.readlines()

print(len(lines))    # 2
```

핵심 정리

- **with** 사용 필수
- 블록 종료 시 파일 자동 종료. `close()` 누락으로 인한 문제 방지
- **encoding="utf-8"** 은 거의 필수
- 한글이 포함된 파일에서 생략 시 윈도우에서 깨짐 또는 오류 발생
- 경로에 역슬래시 주의
- 윈도우 경로는 "C:WWdataWWa.csv" 처럼 두 번 쓰거나, r"C:WdataWa.csv" 형태 사용
- 실제로 CSV는 **pandas**로
- 뒤에서 배울 `pd.read_csv` 한 줄이면 해결

예외 처리 : 오류가 나도 멈추지 않게

파일 100개 처리 중 3개가 깨졌다고 전부 재실행할 수는 없음

```
values = ["1.55", "1.62", "N/A", "1.48"]
gaps = []

for v in values:
    try:
        gaps.append(float(v))
    except ValueError:
        print(f"건너뛸: {v}")

print(gaps)  # [1.55, 1.62, 1.48]

# 여러 오류를 나눠 처리
try:
    f = open("nofile.csv")
except FileNotFoundError:
    print("파일이 없습니다")
finally:
    print("항상 실행되는 부분")
```

핵심 정리

- try 안에서 오류가 나면 except로
- 프로그램이 중단되지 않고 정해진 처리 후 계속 진행
- except를 뭉뚱그리지 말 것
- except: 만 쓰면 모든 오류를 삼켜 실제 문제를 놓침. ValueError처럼 구체적으로 명시
- 실험 데이터에서 자주 만나는 것들
- ValueError(숫자 변환 실패), FileNotFoundError(경로 오타), KeyError(없는 항목)
- 건너뛴 데이터는 반드시 기록
- 누락 사실을 남기지 않으면 이후 결과 설명 불가

감사합니다

이예리

yealee@krikt.re.kr